Secure By Design

EDUCATE. EMPOWER. SECURE THE FUTURE.

COURSE 1.3 / COMPANION WORKBOOK

Configure domain time

The Windows Time hierarchy, the PDC emulator & chrony

Give the air-gapped domain one clock it can trust: make SBD-DC01, the PDC emulator, the reliable source serving its own clock and check it against an approved reference, keep SBD-DC02, Windows members and Linux servers on the hierarchy, and prove the chain with measured offsets.

MATERIALS EDITION 1.0 / COURSE 1.3

Updated 14 October 2026 · sbd-domain-time

NAME _____ / DATE _____

© 2026 Secure By Design Team LLC.

START HERE / YOUR LEARNING PATH

Table of contents

Read each lesson alongside the interactive guide and the video. Use the record tables to write down what you actually observe — the workbook is your evidence, not the guide.

Before you start

01 Plan the domain's time

02 Make SBD-DC01 the domain's time source

03 Bring every system onto the hierarchy

04 Prove the time chain

05 Find the time fault and record the fix

Course review and sign-off

> **HOW TO USE IT** Each lesson page has the same shape: what you will learn, the build steps, the commands that verify them, and a record table. Fill the table from your own environment — never from the sample values.

Before you start

This network has no internet time. SBD-DC01, the PDC emulator, serves time from its own clock, an administrator checks that clock against an approved reference, and everything else follows the domain hierarchy. For the member checks, use a domain-joined Windows test client or member server if you have one (SBD-DC02 shows the same chain if not); the Linux settings in lesson 3 are filed now and applied by the courses that build each Linux server. Thresholds and intervals below are sample decisions — record your own.

Prerequisites

- ☐ Complete courses 1.1 and 1.2: both domain controllers healthy and replicating, and DNS verified on both, so every member can find a domain controller by name.
- ☐ Work at each domain controller's VM console with an authorized Domain Admin account — until course 1.8 builds AdminJumpBox01, that is where administration happens — and have access to the hypervisor settings for both controller VMs.
- ☐ Continue the sbd.test example: SBD-DC01 (10.0.10.10) holds the PDC emulator role and SBD-DC02 is 10.0.10.11, both on VLAN 10.
- ☐ Have an approved time reference that never connects to this network, such as a GPS receiver's display, and a log to record each check.
- ☐ The network's routing and VLAN segmentation are already in place, with a documented service-path register.

What this course builds

- Explain why Kerberos, logs and certificates need agreeing clocks, and plan the hierarchy, reference, thresholds and service path.
- Make SBD-DC01, the PDC emulator, the reliable source serving its own clock, with hypervisor time sync off on both controllers.
- Keep every Windows member and Linux server on the hierarchy, with nothing from Group Policy or a hypervisor overriding it.
- Prove the chain with sources, offsets and events, and fix time faults, a moved PDC emulator role included.

Systems in scope

RECORD	YOUR OBSERVATION
SBD-DC01 — name, address, OS build	
SBD-DC02 — name, address, OS build	

© 2026 Secure By Design Team LLC.

LESSON 01 / ABOUT 25 MIN

Plan the domain's time

WHY THIS MATTERS Every sign-in, log entry and certificate in this network depends on clocks that agree, and an air-gapped network has no internet time to lean on. Learn why time matters here and how the Windows Time hierarchy carries one clock from SBD-DC01 to every system, then write down the reference, the thresholds and the service path you will hold the domain to.

In this lesson you will

- Explain why Kerberos, log correlation, certificates and replication need clocks that agree.
- Trace the Windows Time hierarchy from the forest root's PDC emulator to every member.
- Record the reference, the thresholds, the fallback and the service path the domain's time is held to.

Follow the build

1. Start with why the clocks must agree. Kerberos refuses a request whose timestamp differs from the domain controller's clock by more than the domain's tolerance — five minutes by default — so drift turns into failed sign-ins and failed replication. The logging courses (1.10–1.12) line events up across systems by their timestamps, certificates from the course 1.6 PKI are valid only between two fixed times, and domain controllers need agreeing clocks to replicate accurately.
2. Learn the hierarchy that carries one clock everywhere. The PDC emulator of the forest root domain is at the top; every other domain controller takes its time from the PDC emulator, and every member server and workstation from the domain controller that authenticated it — the NT5DS client type every domain member uses by default. sbd.test is the forest's only domain, so SBD-DC01 is the top and SBD-DC02 follows it, and each Windows member's time arrives signed with its computer's Net Logon session key.
3. Face the air gap. With no internet or outside NTP server, SBD-DC01 serves time from its own clock and is the reference by declaration, so name an approved reference that never connects to this network, such as a GPS receiver's display, and write down who compares SBD-DC01 with it, how often, and how a correction is made and recorded. Record the fallback too: if SBD-DC01 is lost for long, SBD-DC02 eventually stops advertising as a time source, so the PDC emulator role moves to it and it becomes the source (lesson 5).
4. Set the thresholds you will hold. Keep every system within one second of SBD-DC01 — the one-second difference DISA's operating-system requirements use for synchronizing with an authoritative source — and SBD-DC01 within two seconds of the reference, checked daily and after every restart, restore or revert, or at an interval its measured drift has shown it stays inside. Five minutes is where Kerberos fails; these thresholds exist to catch drift long before that.
5. Confirm the service path. Rule F09 in Inside the Network permits every domain member to both controllers on UDP 123, and each controller's inbound firewall rule for the Windows Time service accepts it; SBD-DC02's own requests to SBD-DC01 stay inside VLAN 10, where only the host firewalls see them. SBD-REPO01 (course 1.9) is not a domain member, so F09 does not list it yet; course 1.9 adds it.

Check your result

```
Get-ADForest | Select-Object RootDomain, Domains       # sbd.test: the forest root and its only domain
Get-ADDomain | Select-Object PDCEmulator               # expected: SBD-DC01.sbd.test
```

```
# The domain's Kerberos tolerance, from the Default Domain Policy (expected: MaxClockSkew
= 5, in minutes)
Select-String -Pattern MaxClockSkew -Path "\\sbd.test\SYSVOL\sbd.test\Policies\{31B2F340-
016D-11D2-945F-00C04FB984F9}\MACHINE\Microsoft\Windows NT\SecEdit\GptTmpl.inf"
# Each controller's inbound UDP 123 rule, and where each takes its time today
Invoke-Command SBD-DC01,SBD-DC02 {
  Get-NetFirewallPortFilter -PolicyStore ActiveStore | Where-Object { $_.Protocol -eq
'UDP' -and $_.LocalPort -contains '123' } |
    Get-NetFirewallRule | Select-Object DisplayName, Direction, Enabled, Action
  w32tm /query /source }
```

WHAT SUCCESS LOOKS LIKE The plan names the forest root and its PDC emulator, the hierarchy each system follows, the approved reference with who checks it and how often, both thresholds, the fallback, the Kerberos tolerance read from the Default Domain Policy, and rule F09 with each controller's inbound UDP 123 rule.

Your record

Evidence to keep: Keep the time plan with the output behind it: the forest root and PDC emulator, the reference and checking procedure, both thresholds and the check interval, the fallback, MaxClockSkew, rule F09, both controllers' UDP 123 firewall rules, and each controller's time source before you change anything.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

The domain tolerates five minutes of clock difference. Why hold every system to one second?

> **REMEMBER** Kerberos fails past five minutes; the one- and two-second thresholds catch drift long before that. One hierarchy: SBD-DC01, then SBD-DC02, then every member, with Linux servers pointed at both controllers. The reference, its checks, the fallback and rule F09 are written down before anything changes.

Microsoft · Windows Time service

Microsoft · How the Windows Time service works

Microsoft · How to configure an authoritative time server in Windows Server

Microsoft · Kerberos authentication troubleshooting guidance

DISA · STIGs document library

LESSON 02 / ABOUT 30 MIN

Make SBD-DC01 the domain's time source

> **WHY THIS MATTERS** With no outside time to lean on, the domain's clock is whatever SBD-DC01 says, so make that a deliberate, checked configuration. Take the hypervisor out of both controllers' time, mark SBD-DC01 — the PDC emulator — as the reliable source serving its own clock with no external peer, and run the first check against the approved reference.

In this lesson you will

- Configure SBD-DC01, the PDC emulator, as the domain's reliable time source.
- Turn off hypervisor time synchronization for both domain controller VMs.
- Run and record the first check of SBD-DC01's clock against the approved reference.

Follow the build

1. Confirm which domain controller holds the PDC emulator role — SBD-DC01 in this sample. The PDC emulator of the forest root domain sits at the top of the Windows Time hierarchy, so it is the one system you configure; everything else follows it.
2. Turn off the hypervisor's time synchronization for both controller VMs — SBD-DC01 on SBD-HV01 and SBD-DC02 on SBD-HV02 — so no host clock can silently override the PDC emulator. On Hyper-V that is the Time synchronization integration service; other hypervisors have an equivalent guest-tools setting, and some also set the guest clock after a resume or restore, so check those settings too. Inside each guest, w32tm /query /source must no longer name a hypervisor provider such as VM IC Time Synchronization Provider.
3. On SBD-DC01, mark Windows Time as a reliable time source and restart the service. w32tm /config /reliable:yes writes AnnounceFlags 5 — always a time server (0x1) and always a reliable one (0x4) — in place of the automatic default, 10 (0x2 and 0x8). Leave the client type at NT5DS and add no manual peer: the forest root's PDC emulator has nothing above it, so it serves its own clock, and clients accept time from a server that synchronizes with nothing only because it is marked reliable.
4. Read what SBD-DC01 now advertises: source Local CMOS Clock, stratum 1, reference ID LOCL, and a root dispersion of 10 seconds — the default LocalClockDispersion for a local clock, which you leave alone (lesson 3 sets the Linux servers to accept it). Expect Time-Service warning 12, which says the forest root's PDC emulator has nothing above it and will act as the authoritative source: that is the designed state, so record it rather than fix it. nltest /dsgetdc:sbd.test /gtimeserv should return SBD-DC01 with the GTIMESERV flag, the reliable time server Net Logon now advertises.
5. Run the first reference check with lesson 1's procedure: read the approved reference and SBD-DC01's clock at the same moment and record both. Correct only past the threshold, by the measured difference (Set-Date -Adjust), never by typing a time from memory; Security event 4616 records the change, and your log records who made it and why.

Check your result

```
Get-ADDomain | Select-Object PDCEmulator   # expected: SBD-DC01.sbd.test
# Hyper-V example (other hypervisors have an equivalent setting). On SBD-HV01, and the
same for SBD-DC02 on SBD-HV02:
Disable-VMIntegrationService -VMName SBD-DC01 -Name 'Time Synchronization'
Get-VMIntegrationService -VMName SBD-DC01 -Name 'Time Synchronization'   # Enabled: False
# On SBD-DC01 (the PDC emulator): the reliable source, serving its own clock
```

```
w32tm /config /reliable:yes /update
Restart-Service w32time
w32tm /query /configuration   # AnnounceFlags: 5 (Local); NtpClient Type: NT5DS (Local)
w32tm /query /source          # Local CMOS Clock
w32tm /query /status          # Stratum: 1, ReferenceId: LOCL, Root Dispersion: 10s
nltest /dsgetdc:sbd.test /gtimeserv   # DC: \\SBD-DC01.sbd.test; Flags include GTIMESERV
Get-WinEvent -FilterHashtable @{LogName='System'; ProviderName='Microsoft-Windows-Time-
Service'; Id=12} -MaxEvents 1 |
  Format-List TimeCreated, Message
# Reference check: run this as you read the approved reference, and log both readings
Get-Date -Format 'yyyy-MM-dd HH:mm:ss.fff K'
# Only past the threshold, by the measured difference (example: SBD-DC01 two seconds
slow):
# Set-Date -Adjust ([TimeSpan]::FromSeconds(2))
```

WHAT SUCCESS LOOKS LIKE Hypervisor time synchronization is off for both controller VMs; SBD-DC01 reports AnnounceFlags 5 and NT5DS, source Local CMOS Clock, stratum 1 and reference LOCL; nltest returns it with the GTIMESERV flag; event 12 is recorded as expected; and the first reference check is logged inside the threshold, or with its correction.

Your record

Evidence to keep: Record the hypervisor setting for both controller VMs; SBD-DC01's configuration, source and status output; the nltest result; the event 12 text; and the first reference check — both readings, the difference, any correction with its 4616 event, who and when.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Which system is the domain's authoritative time source in this design?

REMEMBER SBD-DC01, the PDC emulator, is the declared source: AnnounceFlags 5, its own clock, no external peer. No hypervisor sets either controller's clock. The first reference check is logged, and any correction was made by the measured difference.

Microsoft · Windows Time service

Microsoft · How the Windows Time service works

Microsoft · Windows Time service tools and settings

Microsoft · How to configure an authoritative time server in Windows Server

Microsoft · Virtualizing domain controllers with Hyper-V

Microsoft · Manage Hyper-V Integration Services

Microsoft · Configure the root PDC with an authoritative time source

LESSON 03 / ABOUT 35 MIN

Bring every system onto the hierarchy

WHY THIS MATTERS One source is only useful if every other clock follows it and nothing quietly overrides that. Leave SBD-DC02 and the Windows members on the domain hierarchy's defaults, keep Group Policy and hypervisors out of their time, and write the chrony configuration every Linux server in later courses uses to follow both controllers.

In this lesson you will

- Keep SBD-DC02 and every Windows member on the domain hierarchy, with nothing from Group Policy or a hypervisor overriding it.
- Write the chrony configuration every Linux server uses to follow both controllers, and explain maxdistance 16.
- Account for the systems outside the domain hierarchy: SBD-REPO01 and the offline root CA.

Follow the build

1. Leave SBD-DC02 and every Windows member on the defaults: client type NT5DS, no manual peer list, and on SBD-DC02 the automatic AnnounceFlags 10, so it serves time without claiming to be the reliable source. w32tm /query /configuration shows each value with where it came from, (Local) or (Policy), and a system someone once set by hand returns with w32tm /config /syncfromflags:domhier /update and a restart of the service.
2. Keep time out of Group Policy. The Windows Time Service policies (Computer Configuration > Administrative Templates > System > Windows Time Service) overwrite the local W32Time settings, so one that names an NTP server or sets the type to NTP pulls its targets off the hierarchy, and one that reaches SBD-DC01 overrides what makes it the source. Leave them Not Configured in every GPO, the STIG GPOs of courses 1.4–1.5 included: the STIGs expect members on the domain hierarchy, and their rule that the PDC emulator take time from an authorized external source becomes a recorded deviation in course 1.5's exception record.
3. Keep hypervisors out of every virtual machine's time, as you did for the controllers. When a later course builds a virtual machine, server or client — SBD-ADMIN-PC01, the Tier 2 seat on SBD-ADMIN-SRV01, included — turn off its hypervisor time synchronization, so the domain stays the only time authority: a Windows guest whose source reads VM IC Time Synchronization Provider, or any other hypervisor provider, is not following the domain.
4. Write the chrony configuration every Linux server uses. In /etc/chrony.conf (RHEL family) or /etc/chrony/chrony.conf (Ubuntu), remove every pool, server and sourcedir line, add server SBD-DC01.sbd.test iburst, server SBD-DC02.sbd.test iburst and maxdistance 16, and add no allow line, because these servers serve time to no one. Domain-joined Linux servers authenticate with Kerberos through SSSD, so the same five-minute limit applies to them; ntpd, where a distribution still uses it, takes the same two server lines and tos maxdist 16.
5. Record why maxdistance 16 is this path's choice. Serving its own clock, SBD-DC01 advertises 10 seconds of root dispersion and SBD-DC02 a little more, while chrony refuses any source whose root distance is above 3 seconds by default, so without the line chronyc sources marks both controllers ^? (unusable). Sixteen seconds is the largest dispersion NTP defines, so the setting admits both controllers

without admitting an unsynchronized source; the time itself is unsigned, which is why F09 limits it to the two controllers.

6. Apply the same file to every Linux server before its first domain join or package update: SBD-REPO01 (course 1.9), which is not domain-joined, so F09's member list leaves it out until course 1.9 adds it; SBD-SPLUNK01 (course 1.12) and SBD-WEB01 (course 1.18) when they are built; and SBD-NESSUS01 when course 1.17, coming soon, arrives. The offline root CA (course 1.6) is the one clock no network reaches: set it against the approved reference before each signing ceremony.

Check your result

```
# On SBD-DC02, and on a Windows test client or member server if you have one:
w32tm /query /configuration | Select-String 'AnnounceFlags|Type:'   # SBD-DC02:
AnnounceFlags: 10 (Local); Type: NT5DS (Local)
w32tm /query /configuration | Select-String '\(Policy\)'           # expected: no output
w32tm /query /source                                                # a controller, never
a hypervisor provider
# A system once set by hand returns to the hierarchy:
#   w32tm /config /syncfromflags:domhier /update; Restart-Service w32time; w32tm
/resync /rediscover
# The Linux chrony file, filed now (/etc/chrony.conf, or /etc/chrony/chrony.conf on
Ubuntu):
#   server SBD-DC01.sbd.test iburst
#   server SBD-DC02.sbd.test iburst
#   maxdistance 16
# ...with no pool, other server, sourcedir or allow lines. Each Linux build later checks:
#   chronyc sources -v     # ^* one controller, ^+ or ^- the other (^? = unusable)
#   chronyc ntpdata | grep -E '^(Remote address|Stratum|Root dispersion|Authenticated)'
#   chronyc tracking       # Reference ID: a controller; System time well inside one
second
```

> **WHAT SUCCESS LOOKS LIKE** SBD-DC02 shows AnnounceFlags 10 and NT5DS and takes its time from SBD-DC01, a Windows member names a controller, no W32Time value shows (Policy), and the chrony file — both controllers, maxdistance 16 and the reason for it — is filed for courses 1.9, 1.12 and 1.18 to apply.

Your record

Evidence to keep: Keep SBD-DC02's and the member's configuration and source output, the empty (Policy) search, the rule that no VM takes its time from its host, and the filed chrony configuration with the reason for maxdistance 16 and the systems that will use it.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	

RECORD	YOUR OBSERVATION
Open items or findings	

Think it through

A member's w32tm /query /configuration shows Type: NTP (Policy). What does that tell you?

> **REMEMBER** SBD-DC02 and the Windows members run on defaults, and nothing shows (Policy). Every VM takes its time from the domain, never from its host. One chrony file — both controllers, maxdistance 16 — serves every Linux server, SBD-REPO01 included.

Microsoft · How the Windows Time service works

Microsoft · Windows Time service tools and settings

Microsoft · Virtualizing domain controllers with Hyper-V

Red Hat · Configuring time synchronization (chrony)

chrony · chrony.conf(5)

RFC 5905 · Network Time Protocol Version 4: Protocol and Algorithms Specification

DISA · STIGs document library

LESSON 04 / ABOUT 30 MIN

Prove the time chain

WHY THIS MATTERS A source you configured is a claim until every link answers to it. Read each system's source and status, measure offsets against the thresholds from lesson 1, and keep the events and readings that let an assessor see the time chain for themselves.

In this lesson you will

- Confirm each link's source and stratum, from SBD-DC01 down to a member.
- Measure controller and member offsets against the thresholds from lesson 1.
- Keep the Time-Service events and readings that make up the time evidence record.

Follow the build

1. Read each link's source and status, from the top down. SBD-DC01 reports Local CMOS Clock at stratum 1, SBD-DC02 reports SBD-DC01.sbd.test at stratum 2, and a member reports a controller at stratum 2 or 3; a member showing Local CMOS Clock, Free-running System Clock or a hypervisor provider is not following the domain (lesson 5).
2. Measure the controllers against each other: w32tm /monitor queries each controller in the domain, marks the PDC emulator, and reports every other controller's NTP offset from it. SBD-DC02 should sit well inside one second; the ICMP line is only a ping, and the NTP offset is the measurement.
3. Measure a member from the member itself: w32tm /stripchart against SBD-DC01 takes timed samples of its offset, and five of them inside one second meet the threshold. On a Linux server later, chronyc tracking reports the same measure as System time, beside a root dispersion of about 10 seconds that is the controllers' advertised error, not an offset.
4. Keep the Time-Service events that tell the chain's story, from the System log: 12 on SBD-DC01 (it serves as the authoritative source), 35 and 37 on the followers (a source chosen, valid time arriving), and 36, 129 and 142 as warnings worth an alert (no synchronization for a long period, no domain source found, a controller no longer advertising as a time source). Security event 4616 records every change to a clock, your reference corrections included; list these for the event set course 1.10 collects.
5. Write the dated evidence record: for each system, its source, stratum and measured offset against its threshold, plus its time zone, since audit time stamps must be in UTC or carry their offset; then the event extracts, the reference check log, and who ran each check. Repeat the offset checks after any change to a controller.

Check your result

```
New-Item -ItemType Directory C:\Evidence\time -Force | Out-Null
$f = "C:\Evidence\time\$env:COMPUTERNAME-$(Get-Date -Format yyyyMMdd)"
# On SBD-DC01, SBD-DC02 and a member:
w32tm /query /source | Out-File "$f-source.txt"
w32tm /query /status | Out-File "$f-status.txt"
w32tm /tz | Out-File "$f-tz.txt"
# On a controller: each controller's NTP offset from the PDC emulator
w32tm /monitor /domain:sbd.test | Out-File "$f-monitor.txt"
# On the member: five timed samples against SBD-DC01
w32tm /stripchart /computer:SBD-DC01.sbd.test /samples:5 /dataonly | Out-File "$f-
stripchart.txt"
# The Time-Service events to keep, and every clock change (4616)
```

```
Get-WinEvent -FilterHashtable @{LogName='System'; ProviderName='Microsoft-Windows-Time-
Service'; Id=12,35,36,37,129,142} -MaxEvents 50 |
  Select-Object TimeCreated, Id, Message | Export-Csv "$f-time-events.csv" -
NoTypeInformation
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4616} -MaxEvents 20 |
  Select-Object TimeCreated, Message | Export-Csv "$f-4616.csv" -NoTypeInformation
```

WHAT SUCCESS LOOKS LIKE Every link reports the expected source and stratum, w32tm /monitor shows SBD-DC02 well inside one second of the PDC emulator, the member's five samples sit inside one second of SBD-DC01, the event extract holds the designed events with every warning explained, and the reference check log is current.

Your record

Evidence to keep: Keep the dated source, status, time zone, monitor and stripchart files from C:\Evidence\ time, the Time-Service and 4616 extracts, the reference check log, and a one-page statement of each threshold beside the result measured against it.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

w32tm /query /source on a member returns Free-running System Clock. What does that mean?

REMEMBER Each link names the right source: Local CMOS Clock at the top, a controller everywhere else. Offsets are measured against one second, not assumed. Sources, offsets, events and the reference log form one dated record.

Microsoft · How the Windows Time service works

Microsoft · Windows Time service tools and settings

Microsoft · w32tm /resync fails because no time data was available

Microsoft · Event 142: The time service has stopped advertising as a time source

Microsoft · Configure the root PDC with an authoritative time source

NIST · SP 800-53 Rev. 5 security controls

LESSON 05 / ABOUT 30 MIN

Find the time fault and record the fix

> **WHY THIS MATTERS** Time faults rarely look like time: a refused sign-in, a replication error, a certificate that is suddenly not yet valid. Measure the offset first, find where the hierarchy broke — a member off its source, a moved PDC emulator role, a restored or reverted VM — then fix the cause, resync safely and record what happened.

In this lesson you will

- Recognize clock skew behind Kerberos, replication and certificate errors, and measure it before changing anything.
- Return a member to the hierarchy, and move the time source with the PDC emulator role.
- Resync safely after a restore or revert, and record the fix with its rollback.

Follow the build

1. Recognize skew by its symptoms. Kerberos errors that name time — KRB_AP_ERR_SKEW, or 'There is a time and/or date difference between the client and server' at sign-in — access-denied errors in replication between the controllers, and certificates reported as not yet valid all point at a clock; if the controllers audit Kerberos failures, Security event 4771 with failure code 0x25 names the client. Measure before you change anything: w32tm /stripchart from the failing system to SBD-DC01 shows the offset.
2. Find out why a member left its source. Local CMOS Clock or Free-running System Clock means no usable source, a hypervisor provider means its host's time sync is on, and a peer name means someone set it by hand; a (Policy) value in w32tm /query /configuration points at a GPO, and events 129 and 36 show how long it has been adrift. Practice on the test client or SBD-DC02: set a manual peer with w32tm /config /manualpeerlist:SBD-DC01.sbd.test,0x8 /syncfromflags:manual /update, note that its time is still right but unauthenticated and off the hierarchy, then put it back with /syncfromflags:domhier.
3. Move the source with the PDC emulator role. When the role moves — a planned transfer, or a seizure after SBD-DC01 is lost — run w32tm /config /reliable:yes /update on the new holder and w32tm /config /syncfromflags:domhier /reliable:no /update on the old one, restart Windows Time on both, move the reference check to the new holder, and confirm that nltest /gtimeserv names it. Otherwise the old holder still advertises itself as the reliable source and the domain's time can keep flowing from a controller that is no longer the PDC emulator; practice only if you can move the role back the same day, because later courses expect SBD-DC01 to hold it.
4. Treat a restored or reverted VM as a clock problem as well as a directory one. It can come back with the clock it had when it was saved: members correct themselves, a domain controller corrects at most 48 hours each way (MaxPosPhaseCorrection and MaxNegPhaseCorrection) and logs an event instead, and SBD-DC01 has nothing to correct it at all, so every member follows its old time. After any restore or revert of SBD-DC01, compare it with the reference before you trust it.
5. Resync safely. On Windows, w32tm /resync /rediscover finds the source again and resynchronizes; on Linux, sudo chronyc makestep steps the clock once a controller is selectable. When an offset is beyond what the service will correct, set the clock close by hand first, correct SBD-DC01 only by the measured difference, and never point a system at an outside or manual source to make an error go away.
6. Record the fix like any other change: the symptom and when it started, the first offset measured, the cause, what you changed, the evidence that the chain is whole again — source, offset and events —

and the rollback. A moved role also updates the time plan: who now runs the reference check, and the reverse procedure for when the role returns.

Check your result

```
# Measure before changing anything (on the failing system)
w32tm /stripchart /computer:SBD-DC01.sbd.test /samples:3 /dataonly
w32tm /query /source
w32tm /query /configuration | Select-String 'Type:|NtpServer:|\(Policy\)'
Get-WinEvent -FilterHashtable @{LogName='System'; ProviderName='Microsoft-Windows-Time-
Service'; Id=36,129} -MaxEvents 10 |
  Format-Table TimeCreated, Id -AutoSize
# Practice fault, then the fix (test client or SBD-DC02)
w32tm /config /manualpeerlist:SBD-DC01.sbd.test,0x8 /syncfromflags:manual /update
w32tm /query /source            # SBD-DC01.sbd.test,0x8: a manual peer, off the hierarchy
w32tm /config /syncfromflags:domhier /update
Restart-Service w32time
w32tm /resync /rediscover
w32tm /query /source            # a controller again
# If the PDC emulator role moves to SBD-DC02:
#   SBD-DC02: w32tm /config /reliable:yes /update; Restart-Service w32time
#   SBD-DC01: w32tm /config /syncfromflags:domhier /reliable:no /update; Restart-Service
w32time
nltest /dsgetdc:sbd.test /gtimeserv   # must name the current PDC emulator
# Linux: sudo chronyc makestep, then chronyc tracking
```

> **WHAT SUCCESS LOOKS LIKE** The practice fault was found from the system's configuration before anything changed, the system names a controller again with Type NT5DS and an offset inside one second, nltest names the PDC emulator as the reliable source, and the record explains the cause rather than only showing a passing result.

Your record

Evidence to keep: Keep the fault record: the symptom and start time, the first offset, the source and configuration output before and after, the events, the change and its rollback, and the final offsets; after a role move, the updated time plan.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

A member's source reads VM IC Time Synchronization Provider. What is wrong?

> **REMEMBER** Measure the offset first; a symptom only tells you which clock to look at. A member off its source, a moved role and a restored VM each have their own fix. Every fix is recorded with cause, evidence and rollback, and the time plan follows the role.

Microsoft · Windows Time service tools and settings

Microsoft · How to configure an authoritative time server in Windows Server

Microsoft · Virtualizing domain controllers with Hyper-V

Microsoft · Kerberos authentication troubleshooting guidance

Microsoft · w32tm /resync fails because no time data was available

Red Hat · Configuring time synchronization (chrony)

Course review and sign-off

Before moving on, confirm each lesson left evidence behind — and that what remains open is written down, not remembered.

Lesson completion

LESSON	EVIDENCE FILED	DATE
01 Plan the domain's time	☐ yes	
02 Make SBD-DC01 the domain's time source	☐ yes	
03 Bring every system onto the hierarchy	☐ yes	
04 Prove the time chain	☐ yes	
05 Find the time fault and record the fix	☐ yes	

Open items carried forward

Next in the learning path

That completes Configure domain time. Course 1.4, Measure against the STIG, scans both controllers — and meets the one rule this design answers with a declared, checked clock.

> **KEEP IT HONEST** Lesson completion records learning activity; it is not a system authorization decision. Your observations, not the sample values, establish what happened in your environment.